

The YASO Model: YOSO without Erasure

Jesper Buus Nielsen¹, Ivan Damgård¹, and Sannidhi V. Hebbar²

¹ Aarhus University, Denmark
{jbn,ivan}@cs.au.dk

² Indian Institute of Science, India
sannidhiv@iisc.ac.in

Abstract. The YOSO paradigm protects against a strongly adaptive adversary. Each ephemeral role prepares one message, erases all remaining state, and only then speaks. We ask what remains possible when erasure is unavailable and speaking instead exposes the sender’s complete state. At first sight, such a model appears useless. Nevertheless, it has a different irreversible resource: a machine may choose never to speak, in which case the secret state assigned to it can remain hidden. We call the resulting model YAMSO, for “You At Most Speak Once.” This preliminary article introduces the model, gives a finite two-phase garbled-circuit blueprint, and develops a candidate prefix-preserving garbled-circuit factory for ongoing computation. The latter uses one-use common-input certificates to bind a copyable obfuscated stream to the canonical ledger history. We isolate its security into three explicit replacement lemmas. A horizon-free public-program realization and online random-oracle programming are the two main remaining gaps.

Keywords: adaptive security · YOSO · erasure-free protocols · one-time roles · garbled circuits · indistinguishability obfuscation

1 Introduction

The YOSO (You Only Speak Once) model was introduced as a way to obtain secure multiparty computation against a very strong adaptive adversary [7]. A protocol role is assigned to an ephemeral party which prepares one message, deletes the secret state used to prepare it, and then sends the message. Only at this point does the role reveal itself. Corrupting the sender after publication is therefore too late: the message is already available and the secrets that produced it no longer exist. Later work has developed this anonymous, ephemeral-role paradigm in several directions; the erase-before-send discipline remains central.

This paper starts from the deliberately unfriendly question: what if deletion is impossible? The restriction might be legal—a regime may require state to be retained for later inspection—or technological. For example, a party running in a cloud environment might fear that speaking identifies its machine and enables the cloud operator to inspect a residual snapshot despite an attempted deletion. We capture the extreme version of this concern by declaring that whenever an ordinary party speaks, its complete state leaks to the adversary.

The immediate answer seems to be “obviously impossible.” Eventually all parties relevant to the computation will speak and hence reveal everything. This conclusion overlooks one asymmetry. Speaking is optional. A machine may remain silent forever, in which case its secret state may stay hidden. We therefore place mutually exclusive pieces of state behind different public-key roles. The selected roles speak and are expended; the unselected roles never speak. In this sense, silence replaces erasure as the one-time resource.

We call the resulting setting *YAMSO*: You At Most Speak Once. In a *YAMSO* execution, a synchronous ledger maintains a growing ordered list of public keys and messages. The public-key index determines the protocol role. For the preliminary model, each newly registered computation role is corrupted independently with probability p , whereas named application parties retain the ordinary corruption interface. An honest role keeps its secret key inside a protected ledger interface until its scheduled opportunity to speak. Once a computation role speaks, its key and complete role state may become public; its message is already irrevocable and the role is never used again.

The intended real-world interpretation has a much larger population of apparently interchangeable machines. The adversary compromises a fixed fraction of that population, while fresh clients and role states are placed on machines randomly and without exposing the placement before a machine is corrupted

or speaks. Random placement then turns the fixed compromised set into an approximately random intersection with each protocol committee. The Bernoulli experiment in this paper deliberately collapses that mechanism into a simple model; it does not model mobile corruption.

Finite feasibility. The basic two-phase blueprint isolates why the model need not be vacuous. For each garbled input wire, the two structural labels are placed behind disjoint committees, their semantic order is hidden, and an activation token causes only one committee to speak. Its complete exposed state contains shares of a label that is becoming public anyway. Honest members of the sibling committee remain silent forever and protect the unused label. A missing or malformed input closes onto a fixed structural branch whose hidden semantic value is a uniform default bit.

Ongoing computation. The difficult part is not evaluating another garbled circuit; it is preserving the hidden correlation between the current state-label pair and every later stretch without giving a speaking linker both labels. We propose a finite public *oblivious garbled-circuit sampler* (OGCS), implemented conjecturally by an obfuscated streaming program. It fixes one public transition circuit, receives a growing sequence of certified public inputs, and emits exactly prefix-preserving garbled stretches. The output-state label pairs of one stretch are literally the input-state pairs of the next.

A copyable obfuscated program can otherwise be evaluated on invented public keys and forked ledger histories. Our second use of silence is therefore a one-time common-input certificate. Each bit of a high-distance encoding is represented by two one-way preimages placed in different one-use roles. The role matching the canonical message speaks; its opposite remains silent. After the correct certificate is public, the speakers may reveal everything without enabling a conflicting certificate. This observation removes the long-lived certification key we initially appeared to need. The ledger still provides a common immutable view, but it need not hold a signing secret.

This is an instance of the model changing the design intuition. If a single certifier speaks, its leaked signing key destroys future authenticity. The natural reaction is to declare certification impossible. YAMSO instead asks which secret holders need never speak. The resulting construction is closely related to known high-distance Lamport signatures, but its use as persistent common truth under total post-speech exposure is the mechanism we need here.

Status and proof strategy. The ongoing construction is intentionally presented as a candidate, not a theorem. We modularize its analysis by three whole-process replacement lemmas: canonical one-use certification, programmable prefix-preserving OGCS, and two-stage finite garbling/readout with full speaker exposure. This avoids a naive UC-hybrid decomposition whose interfaces would export internal keys to the environment. The first version fixes corrupt input providers before setup and has public outputs. Two central realization gaps remain: the cited general-purpose ordinary-iO-based succinct TM-iO is bounded-input rather than horizon free, and predictable random-oracle points do not yet support one online simulator. A hidden nonce prepared one stretch ahead is the intended lift for the second gap.

Relation to prior work. Our starting point is YOSO [7]. Anonymous blockchain committees tolerating a corruption rate around one quarter are known, but their secret handoff relies on erasures [2]. Blockchain one-time programs already demonstrate conditional release of one garbled input-label vector on an immutable ledger [9]. No-erasure hidden committees also appear in communication-local MPC, although there the selected committee runs an ordinary multi-round computation [3].

For certification, Katz and Vaikuntanathan give a direct high-distance Lamport precedent [12], and Kelsey, Lang, and Lucks study distributed hash-based signing and the disjoint-honest-trustees one-time-key-reuse problem in the absence of broadcast [13]. Neither result states our one-shot common-input property under post-speech full exposure. For the streaming component, relevant technical precedents include universal samplers, cryptographic iterators, succinct and reusable garbling, TM obfuscation, and streaming functional encryption [11,14,16,8,10]. We use these as component precedents rather than claiming that an existing theorem already gives the YAMSO construction.

2 Preliminaries: What Must Be Fixed

This section is presently a roadmap. The final paper should reproduce every definition whose details affect the YAMSO claim, while importing routine cryptographic syntax and standard security definitions by reference.

2.1 Conventions

We write λ for the security parameter, $[n] = \{1, \dots, n\}$, and $\{0, 1\}^*$ for the set of finite bit strings. All adversaries and protocols are probabilistic polynomial time unless explicitly stated otherwise, and $\text{negl}(\lambda)$ denotes a negligible function. Protocol identifiers, epochs, wire identifiers, public-key indices, ledger positions, and object types must be included in every cryptographic domain separator. A computation key is assigned to exactly one canonical protocol role.

2.2 Definitions that belong in this paper

The following objects are model-specific and should eventually be defined in full.

Ledger. One synchronous, append-only list L seen identically by every party. It begins with a generic CRS, grows by registration and scheduled message entries, and performs no protocol-specific validation. It has no cryptographic certification key.

Growing key registry. An adversarial registration request creates a computation PID equal to its fresh public key. A registration from an existing application PID creates and publicly binds a protected key to that named party. Neither request contains a protocol role; registration order determines roles at the protocol level.

YAMSO turn. A public initialization parameter `WhosTurn` selects one unspent PID from the current L . The selected party either produces one message or receives an explicit `NoMsg` entry and is then spent forever.

Protected local evaluation. An honest scheduled party supplies an algorithm A privately to the ledger, which evaluates $A(sk, L)$ without first returning sk to the party. This lets an ordinary role reveal its entire key and an input role reveal only an input-dependent function of its state through the same generic interface.

Two corruption rules. Each computation PID is independently corrupt with probability p when registered and, if honest, cannot later be corrupted. Named application parties have the ordinary adaptive corruption interface in the model. The first construction statement restricts their corrupt set to be fixed before setup.

Closed input slot. At a protocol-defined cutoff, the public ledger prefix determines one immutable input record. Public branch-indexed one-way tags determine whether it contains exactly one valid activation token. A missing, ambiguous, or malformed record selects the structural default branch; late messages are ignored.

Common-input certificate. Fresh one-use roles hold opposite one-way preimages for the coordinates of an encoded digest. Honest roles compute the value to be certified from the same closed ledger cut, rather than signing a caller-supplied string. Verification permits enough missing openings for liveness but must rule out a second accepted digest.

OGCS prefix. A fixed finite public object is evaluated on complete finite histories. Outputs on every accepting history are literal prefixes. A history that first differs at some accepted command must not reveal the hidden logical state of the canonical history; a rejected history never extends again.

2.3 Primitives that can largely be imported

Standard definitions of pseudorandom functions, collision-resistant and one-way hashing, commitments, secret sharing, public-key encryption, and circuit garbling need not be repeated in full. We should state the exact variants used and cite them. The following interfaces require particular care.

Role-state encryption. Role membership is public in the preliminary model, so ordinary multi-user encryption can protect the state placed under each registered key; recipient anonymity is not required by this abstraction. The semantic association between the two structural committees for an input wire must nevertheless remain hidden. Each key has one canonical role packet. Off-path evaluations may address further packets to the same key, and later disclosure decrypts them; the OGCS isolation lemma must make all such plaintexts independently rooted fork material.

Verifiable threshold sharing. The construction needs privacy below the reconstruction threshold, reconstruction from honest shares, and public rejection of malformed shares or keys. Public verification data must not disclose the semantic branch permutation.

Two-stage garbled circuits. Preparation fixes external-input label pairs, tokens, and role packets before inputs arrive. Completion occurs only after input closure and must support prescribed active external and carried-state labels, prescribed public output, hidden next-state label pairs, and consistency with every later-exposed selected role. This causal interface is stronger than a bare one-shot privacy statement.

High-distance one-time opening. The certificate uses the distance idea from Lamport-style leakage-resilient signatures [12]. The paper must prove its own partial-opening liveness and non-equivocation statement under random role corruption and complete post-speech exposure, including an explicit code-rate-versus-tail calculation.

Succinct TM-iO and puncturing. For a declared input-length bound, the candidate OGCS can use a compact obfuscated Turing-machine program with puncturable PRF/KDF state [14,1]. We must state the bound, succinctness, functionality-equivalence condition, and any equal-running-time requirement. The cited general-purpose ordinary-iO-based bounded-input results do not by themselves realize the horizon-free replay interface of section 5.

Delayed backdoor programming. Hofheinz et al.’s adaptive universal sampler provides the hidden-trigger paradigm [11]. Selectively secure pseudorandom puncturable deterministic encryption gives a related formulation [15]. Their theorems do not apply automatically to our stateful syntax; the OGCS replacement lemma must be proved separately.

Statistical binding for iO hybrids. Computational unforgeability does not by itself justify iO between programs that differ on an invalid but mathematically possible input. In its binding setup, a somewhere statistically unforgeable signature makes alternate valid signatures at a selected round and message statistically nonexistent, except with negligible setup probability [6]. Adapting that property to our certificate chain is an open proof obligation.

Proof boundary. The main text uses the generic ledger interface of section 3. Once the API is stable, a detailed $\mathcal{F}_{\text{Ledger}}$ can be placed in an appendix. The target functionality contains only named application parties; the randomly corrupted computation population is implementation machinery to be simulated away. Intermediate cryptographic components are handled by whole-process replacement lemmas, avoiding hybrid interfaces that export their secret trapdoors to the environment.

3 The YAMSO Ledger Model

We describe the ledger interface at the level needed for the finite construction in section 4. Our approach follows the modelling style of [5, Section 2.1]: we state an intentionally strong and simple abstraction, isolate the properties used by the protocol, and postpone its detailed realization. A later version will express this interface as an ideal functionality $\mathcal{F}_{\text{Ledger}}$, in the style of the total-order-broadcast functionality of [4]. There is no reason to fix that UC bookkeeping before the API itself has stabilized.

The ledger is generic. It knows nothing about universal sampling, garbled circuits, committees, input wires, or the meaning of protocol roles. Those belong to the protocol using the ledger.

3.1 Public state and initialization

Fix a security parameter λ , a session identifier sid , a constant $p < 1/3$, a generic setup distribution Setup , a key-generation algorithm KeyGen , and a description of a deterministic protocol-dependent algorithm WhosTurn . The description of WhosTurn is supplied to the ledger as a fixed initialization value; it cannot be changed in response to the execution. The ledger samples

$$\text{crs} \leftarrow \text{Setup}(1^\lambda)$$

and initializes the public append-only list as $L = ((\text{Setup}, \text{crs}))$. The concrete protocol gives meaning to crs ; the ledger does not interpret it.

We assume synchronous computation and an unrealistically strong broadcast abstraction. After an entry is appended, every party sees exactly the same list L at the same time. An append is atomic, cannot be suppressed after it occurs, and fixes the position of that entry. There are no delayed, private, or party-local ledger views. This deliberately idealized assumption removes consensus and network scheduling from the present question.

Apart from its own setup, registration, and timeout records, the ledger does not enforce protocol syntax. In particular, an arbitrary message supplied by a corrupt scheduled party is appended verbatim. The protocol reading L decides which entries are well-formed and ignores malformed ones.

3.2 Two registration paths

The registry grows during the execution. There are two ways to add a key; neither command takes a protocol role as input. The interface imposes no a-priori bound on the registry, although any efficient execution creates only polynomially many keys and every finite protocol instance uses a finite prefix.

Computation-party registration. On a Register request from the adversary, the ledger samples

$$(pk_i, sk_i) \leftarrow \text{KeyGen}(1^\lambda),$$

where i is the next computation-key index, creates an ITM whose PID is pk_i , and appends (Register, pk_i) to L . Independently it samples $c_i \leftarrow \text{Bernoulli}(p)$. If $c_i = 1$, the new computation party is corrupt and sk_i is given to the adversary. Otherwise it is honest, the ledger retains sk_i as protected state, and the adversary cannot corrupt this party later. Registration is indivisible: after requesting it, the adversary cannot discard the public key because it dislikes the corruption outcome.

Application-party registration. A fixed application party with ordinary UC identity P may also input Register. The ledger samples $(pk_P, sk_P) \leftarrow \text{KeyGen}(1^\lambda)$, stores the association $P \leftrightarrow (pk_P, sk_P)$, and appends (Register, P, pk_P) to L . It does not return sk_P to an honest P . There is no Bernoulli corruption experiment for such a party: the adversary may corrupt P adaptively in the ordinary UC sense. If P is already corrupt, or becomes corrupt later, the ledger releases sk_P and its retained record of P 's protected calls to the adversary. The target ideal functionality supplies the simulator with any input or output leakage prescribed upon that corruption.

Each PID registers at most once per session. Application parties are the meaningful principals at the interface of the computation—for example, an account owner supplying an input. Their identity need not and generally cannot be hidden. Computation parties, by contrast, are protocol machinery.

A public protocol-level rule maps registration positions to roles. Thus the seventh computation key may represent the eighth share-release role. This mapping is not an argument to Register and is not interpreted by the ledger.

3.3 Turns and protected local evaluation

At each synchronous step, the fixed algorithm $\text{WhosTurn}(L)$ returns either a registered public key that has not previously had a turn, or $\perp$. It may schedule a computation key pk_i or the registered key pk_P of an application party. A well-formed protocol chooses WhosTurn so that it waits for enough registrations and schedules each required role in the appropriate order. The ledger does not interpret why a particular key is scheduled.

When a key pk is scheduled, its owner has one synchronous opportunity to speak. Write L_{pre} for the list at the start of that turn. An honest owner privately gives the ledger a finite polynomial-time algorithm A . The ledger samples and retains any coins required by A , and evaluates

$$m \leftarrow A(sk, L_{\text{pre}}),$$

using the protected secret key associated with pk . If $m \neq \perp$, it signs $(\text{sid}, L_{\text{pre}}, m)$ using the signing component of sk and appends

$$(\text{Message}, pk, m, \sigma)$$

to L . Neither A nor the unrevealed part of sk is output. The ledger retains the algorithm and its coins as the protected call record rather than modelling an erasure.

If the owner is corrupt, the adversary knows sk and may supply any pair of bit strings (m, σ) during the turn. The ledger appends (Message, pk, m, σ) verbatim without checking a signature, a key relation, or any other protocol-specific predicate. If an honest A returns $\perp$, or if no message is supplied by the end of the turn, the ledger appends (NoMsg, pk). This timeout is immediately visible to everyone and lets

WhosTurn advance past a silent role. After either record is appended, pk is spent and is never scheduled again.

For an honest nonempty message, protected evaluation, signing, and append are one atomic operation. The adversary sees the result only after it is irrevocably in L . The ledger does not give an adversarial scheduler an opportunity to inspect the result and then suppress it.

We assume that KeyGen may provide a public relation $\text{KeyMatch}(pk, sk)$. A protocol that uses disclosure of sk as its message can then check that the disclosed key belongs to the scheduled public key. If the chosen key system has no such relation, the ledger does not manufacture one: the protocol must tolerate the missing check or choose different key syntax.

3.4 How protocols use the interface

The following examples explain the intended use without adding commands to the ledger.

Ordinary computation role. From crs and the public list, all parties may derive the same protocol object, including a ciphertext C_{pk} under a scheduled public key. This object need not itself be an entry of L . The honest role supplies an algorithm A which derives C_{pk} , decrypts its local state with sk , examines L , and returns sk if the role is activated and $\perp$ otherwise. If it returns sk , everyone can check KeyMatch , decrypt C_{pk} , and continue the public computation. Thus publishing the key is the role's only message and models complete post-speech exposure. If A returns $\perp$, the secret key and encrypted role state remain hidden forever.

For this interpretation, every secret used by an ordinary role must be recoverable from sk and public protocol data. A protocol that gives the role additional hidden auxiliary state cannot claim that publishing only sk models leakage of its complete state.

Strong input role. An application party P embeds its private input in A and causes the ledger to append only the prescribed value $A(sk_P, L)$ and its signature. For example, A may decrypt two activation tokens and return the one selected by an input bit. If P remains honest, neither sk_P , the unused token, nor the input-dependent algorithm is revealed. If P is corrupted, the adversary receives its ordinary UC state together with the key and protected call history retained by the ledger; correspondingly the simulator receives the input leakage specified by the target functionality. Corruption after the turn does not permit a replacement message because pk_P is already spent.

The present finite construction has public output. A later private-output variant may add a dual protected-read operation which evaluates a function of (sk_P, L) and returns only its result privately to P ; no such operation is needed here.

3.5 Random corruption and finite committees

The random coins apply only to computation-party registrations. Let C be any size- k computation committee selected from registration indices without using the hidden coins $(c_i)_i$, and let $X = |\{i \in C : c_i = 1\}|$. Then $X \sim \text{Binomial}(k, p)$. At the concrete choice $p = 1/4$, Hoeffding's inequality gives

$$\Pr[X \geq k/3] \leq \exp(-k/72).$$

If a finite execution uses q such committees, a union bound gives

$$\Pr[\neg \text{Good}] \leq q \exp(-k/72),$$

where Good is the event that every committee has at most $t = \lfloor (k-1)/3 \rfloor$ corrupt members. Hence $k = \Theta(\lambda + \log q)$ makes the bad event negligible for every polynomial-size finite execution. Application parties do not enter this calculation.

3.6 Interpretation and limitations

The preliminary experiment should not be read as saying that a real adversary naturally corrupts machines by tossing independent coins. The intended real-world picture has a large population of

apparently interchangeable machines and an adversary that may choose any fixed p fraction of that population to compromise. Fresh computation clients, keys, and role states are placed on machines at random, without revealing the placement before a machine is corrupted or speaks. A key’s logical role may be public even though the physical machine carrying its secret state is not linkable from the ledger. Random placement therefore turns the adversary’s fixed physical set into an approximately random corrupt subset of each computation committee. The independent Bernoulli coins in the present model collapse this placement argument into the cleaner statement that each computation PID is corrupt independently.

This is a fixed-population, non-mobile abstraction. Once a computation PID is sampled honest it cannot later be corrupted. An adversary that can repeatedly replace the corrupted quarter of the physical population is strictly stronger and is not modelled here. Named application parties are different on purpose: an account owner or other designated input provider is not plausibly hidden by random placement, so it retains the ordinary adaptive corruption interface. The random computation corruptions are an implementation assumption to be simulated away, not a corruption rule for the target application.

The statement that the ledger stores sk and evaluates $A(sk, L)$ is likewise an ideal-interface device, not a proposal that a blockchain hold users’ secret keys or execute their private code. In the intended implementation, a machine stores its own key and locally computes the one value it places on the public board. Moving that local step behind the ledger interface prevents an honest party’s secret key from becoming an output visible to the UC environment. For a computation role, publishing sk is an explicit way to model that speaking exposes all of its encrypted role state; for a named application party, only the prescribed value is published unless the party is actually corrupted.

This abstraction is intentionally stronger and cleaner than such a system. It assumes honest key generation, independent placement, no selective abort of an unfavourable registration, instantaneous common knowledge of L , unpreventable honest appends, exact synchronous timeouts, protected evaluation under a secret key, and a protocol-dependent turn function fixed at initialization. It does not model consensus, denial of registration, network partitions, grinding, compromise correlations, or the mechanism that places a computation key on a real machine. These assumptions must not be mistaken for properties already obtained by a concrete blockchain.

Finally, the computation PIDs and their random corruption coins exist only in the real or ledger-hybrid execution. The target ideal functionality has the usual application interface: named input and output parties, their prescribed adaptive corruptions, and the intended outputs. It contains no committees, randomly corrupted computation population, garbled circuits, or *WhosTurn*. A security proof must simulate all of that protocol machinery from the target functionality’s leakage.

The final paper may keep only this interface and interpretation in the main text. Once the API is stable, an appendix can formulate $\mathcal{F}_{\text{Ledger}}$ in UC notation and make precise ITM creation, private delivery and retention of A , synchronous timeouts, and corruption bookkeeping. Those details should express the API above rather than determine it.

4 The Basic Finite Construction

We illustrate the central idea for one evaluation of a public Boolean circuit F . The input providers are named application parties and the output is public. Computation roles are assigned to registered computation keys by a public rule. The construction uses the ledger only through the generic API of section 3; garbling, role assignment, encryption, slot closure, and message validation are protocol operations. Our first proof target fixes the corrupt set of named input providers before setup. The adaptive interface of section 3 is retained as a later target.

This section is a bounded, two-phase blueprint. It cleanly isolates the use of silence, but does not by itself instantiate the sampler that correlates its pre-input and post-input phases. That requirement becomes property Z in section 6; the streaming candidate of section 5 implements the same schedule repeatedly.

4.1 Public structural branches

Fix a ledger prefix L_0 containing the registered input providers and at least $2mk$ fresh computation keys, where m is the number of input wires. For each wire w , the public role map selects two disjoint size- k

committees C_0^w and C_1^w . All $2m$ committees are mutually disjoint, and no computation key is assigned another canonical role. Their subscripts are *structural branch indices*; they do not denote semantic bits.

For concreteness, the generic setup distribution may place a finite sampler parameter U_{fin} in crs . Before inputs arrive, its preparation phase samples for each input wire

$$(K_0^w, K_1^w), \quad (T_0^w, T_1^w), \quad \pi_w \leftarrow S_2.$$

The garbling uses the semantic assignment

$$b \mapsto K_{\pi_w(b)}^w.$$

Equivalently, one may view it as garbling the structurally reindexed circuit

$$F_{\pi}(z) = F(\pi_1^{-1}(z_1), \dots, \pi_m^{-1}(z_m)).$$

The pre-input public object Prep_F contains the encrypted role packets described below, but not yet the garbled tables. The complete label pairs and semantic maps remain hidden in the finite sampler's logical state.

For public token validation, preparation publishes the branch-indexed tags

$$V_s^w = h_{\text{tok}}(\text{sid}, w, s, T_s^w), \quad s \in \{0, 1\}.$$

Tokens have high min-entropy, and h_{tok} is assumed one way and collision resistant on the domain-separated token inputs. Thus the tags allow everyone to recognize structural branch s without revealing the token in advance or the semantic value of that branch.

Label K_s^w is verifiably $(t+1)$ -out-of- k shared among C_s^w , where $t = \lfloor (k-1)/3 \rfloor$. For each member with public key $pk_i \in C_s^w$, Prep_F contains

$$D_i = \text{Enc}_{pk_i}(w, s, T_s^w, \text{share}_{i,s}, \text{verification data}).$$

It similarly contains, under the registered key of the application party providing wire w , an encrypted packet specifying the map $b \mapsto T_{\pi_w(b)}^w$. Packet lengths, ordering, verification data, and ciphertext distributions are symmetric in the two structural branches. Neither π_w nor the semantic bit represented by a public committee is revealed.

The fixed **WhosTurn** waits until L_0 exists and then schedules the provider keys and role keys in a canonical order. Registration itself has no role argument.

4.2 Input activation and irrevocable default

When the provider for wire w is scheduled, an honest provider with input x_w supplies a protected algorithm A_{w,x_w} to the ledger. The algorithm derives Prep_F , decrypts the provider packet using sk_P , and returns the tagged structural token $T_{\pi_w(x_w)}^w$. The ledger appends this output without revealing sk_P , the other token, or the input-dependent algorithm.

A corrupt provider may instead put any string in its turn. At a fixed cutoff the protocol derives one irrevocable close record m_w . Define

$$\text{Valid}_w(m) = s \iff h_{\text{tok}}(\text{sid}, w, s, m) = V_s^w$$

for exactly one $s \in \{0, 1\}$; otherwise set $\text{Valid}_w(m) = \perp$. A **NoMsg** record, malformed string, ambiguous match, and late entry all give $\perp$. The effective structural branch is

$$s_w = \begin{cases} \text{Valid}_w(m_w), & \text{Valid}_w(m_w) \neq \perp, \\ 0, & \text{Valid}_w(m_w) = \perp. \end{cases}$$

Because π_w is uniform and hidden, default branch 0 has semantic value $\pi_w^{-1}(0)$, a uniform bit. A corrupt provider that has opened its own packet knows the semantic map and may choose its effective input, just as a corrupt ideal input party may.

The roles, not a protocol-aware ledger, implement this rule. Committee C_s^w speaks precisely if $\text{Valid}_w(m_w) = s$, or if $\text{Valid}_w(m_w) = \perp$ and $s = 0$. In particular, C_0^w can distinguish a valid branch-1 token from a malformed token using the public tags and therefore does not speak in the former case. Closure must precede the default release: otherwise a malformed message followed by a late valid token could expose both labels.

4.3 Conditional key release and evaluation

Each computation member is scheduled once. Its honest protected algorithm derives its canonical D_i , decrypts it with sk_i , checks the public close record and token tags, and returns sk_i exactly under the rule above; otherwise it returns $\perp$. The selected committee therefore reveals its keys, while every honest member of the sibling committee produces only a **NoMsg** timeout and keeps its key hidden forever.

The ledger performs no validation. Protocol observers ignore a purported key unless $\text{KeyMatch}(pk_i, sk_i)$ holds, decrypt D_i themselves, and reject malformed shares using the public verification data. Conditioned on **Good**, the selected committee has enough honest members to reveal at least $t + 1$ valid shares despite corrupt withholding. Everyone reconstructs exactly one active structural label: $K_{\pi_w(x_w)}^w$ for a valid honest input and K_0^w for a defaulted input.

Only after all input slots close does the completion phase emit the public garbled tables $\tilde{F}$ and output-decoding information, using the label pairs and assignment in (4.1). All parties evaluate $\tilde{F}$ from the reconstructed active labels and decode the public output. This order is important for the intended straight-line simulation: the ideal public output is available before the tables must be fixed. Publishing the tables before input closure would instead require an online or adaptively simulatable garbling primitive.

Ordinary garbling decoding is sufficient. A protocol needing a private bit y for an honest recipient can, at the application layer, let the recipient input a fresh pad r and publicly output $y \oplus r$; version 1 makes no claim for later adaptive corruption of that pad holder.

4.4 Why complete exposure need not be fatal

Publishing sk_i exposes the selected role's complete retained state, including its packet plaintext, local coins, and call record. The exposed share belongs to a label reconstructed publicly anyway and gives no share of the disjoint sibling label. In the sibling committee, at most t members are corrupt on **Good**, while every honest member remains silent; threshold privacy therefore protects the unselected label.

The hidden permutation is necessary in this public-role model. Without it, the speaking committee would reveal the input. With it, the public structural branch has no fixed semantic meaning, and garbling privacy handles the remaining dependence on the input.

The static-corruption restriction on named providers is used here. A provider corrupt before setup controls its slot; an honest provider is never later asked to explain the encrypted unused token or its hidden semantic map. An adaptive extension will require an equivocal or non-committing treatment of the provider packet.

This remains a blueprint rather than a theorem. A proof must combine the two-stage finite-sampler property, multi-user encryption, token-tag security, verifiable-sharing privacy and robustness, garbling privacy, and the good-committee bound. It must also account for retrospective decryption. One-role-per-key gives one canonical packet, not one ciphertext globally: adversarial evaluations of a copyable sampler may address further fork packets to the same pk_i . Canonical-root isolation must ensure that every such plaintext is independent fork material containing no canonical inactive label; the readout part of property Z must simulate all openings.

A gate-by-gate variant. For intuition, garbling can be replaced by an expensive information-theoretic construction. For every NAND gate and public structural branch corresponding to a pair of input tokens, a separate committee holds shares of the corresponding output token. Only the committee whose branch matches the public inputs reveals its keys. This makes the role of silence especially transparent, at the cost of a large setup and many registered computation parties.

5 A Streaming Garbled-Circuit Factory

The finite construction samples one bounded garbling. For ongoing computation we now describe a more specific target than a general universal streamer: an *oblivious garbled-circuit sampler* (OGCS) for one fixed public transition circuit

$$F(s_e, a_e) = (s_{e+1}, y_e),$$

where s_e is carried state, a_e is fresh input, and y_e is public. One application of F is a *stretch*. The desired public object has a finite description and accepts arbitrarily long finite histories; in particular, its interface has no setup-time horizon.

This section gives a candidate construction and a precise interface against which a proof can be written. It is not yet a theorem. The cited general-purpose ordinary-iO-based succinct TM-obfuscation results do not immediately provide the horizon-free interface, and the final online random-oracle programming step also remains open in this first version.

5.1 The prefix interface and its schedule

Setup publishes a finite parameter U_{ogcs} fixing F and containing a fresh identifier uid . The public algorithm is evaluated on a complete finite input history:

$$\text{OGCS}(U_{\text{ogcs}}; I, (M_0, \Sigma_0, \mathbf{W}_0), \dots, (M_e, \Sigma_e, \mathbf{W}_e)) \longrightarrow D_e.$$

The physical object is copyable and stateless; its hidden *logical* state is reconstructed by replaying the input history. On every accepting history its outputs are exactly prefix preserving,

$$D_{e-1} \preceq D_e,$$

and repeated evaluation on the same history gives the same bit string. A failed check enters an absorbing reject state and emits no further extension. The certificate is an acceptance witness, not stream input: two histories are called command-equivalent when they have the same I and the same sequence of $(M_e, \mathbf{W}_e)$ values, even if they use different valid encodings of Σ_e . The program normalizes away certificate bytes after verification. A different initialization or non-equivalent accepted command history is a fork and must not expose the hidden state of the canonical history.

The initialization input I produces a preparation object Prep_0 but no garbled table or application output. For a fixed public initial state s_0 , it also publishes exactly one active initial-state label per state wire as part of Prep_0 . The first certificate key VK_0 is likewise a component of Prep_0 . In general, Prep_e is public before the input slots of stretch e open. It contains

- encrypted activation tokens and verifiable label shares, together with public token-validity tags, for the fresh external-input wires of stretch e ; and
- encrypted branch secrets and public verification material for a fresh one-time certificate used to close that stretch.

The hidden plaintext material underlying Prep_e was fixed by the preceding accepted extension, or by I when $e = 0$. For notation set $D_{-1} := \text{Prep}_0$.

The order within a stretch is important.

1. Input providers post activation tokens. At a deterministic ledger cut every slot is closed and the selected input committees reconstruct one active external-input label per wire. Missing or malformed slots use the publicly detectable random-default rule of section 4.
2. The registration window for stretch $e + 1$ closes. The canonical ledger cut determines one ordered vector PKBatch_{e+1} . If the required number of fresh keys is unavailable, the extension waits.
3. Everyone computes the finite vector $\mathbf{W}_e$ of random-oracle values specified below. This vector may be transported by any party and need not be stored on the ledger; it has no authority of its own.
4. The certificate roles derive one canonical checkpoint M_e from the closed ledger view and reveal a certificate Σ_e for it.
5. The accepted triple $(M_e, \Sigma_e, \mathbf{W}_e)$ appends

$$\text{GC}_e, \quad \text{Dec}_e, \quad \text{Prep}_{e+1}$$

to the cumulative OGCS output.

6. Everyone evaluates GC_e . This gives the public output y_e and one active label on every next-state wire.

Publishing the garbled tables only after input closure lets the straight-line simulator obtain the public ideal output before it must fix those tables. A variant publishing the complete garbling earlier would require a stronger online or adaptively simulatable garbling notion.

5.2 A one-time common-input certificate from silence

The ledger has no signing key. Persistence instead comes from fresh YAMSO roles that certify a value they compute from their common ledger view.

Let $\text{Code} : \{0, 1\}^\kappa \rightarrow \{0, 1\}^n$ be a binary code of relative distance δ , let h_{sig} be collision resistant, and let f be one way. For certificate epoch e , the hidden generator samples independently

$$A_{j,b}^e \leftarrow \{0, 1\}^\lambda, \quad Y_{j,b}^e = f(\text{sid}, e, j, b, A_{j,b}^e)$$

for $(j, b) \in [n] \times \{0, 1\}$. The public verification material is $\text{VK}_e = (Y_{j,b}^e)_{j,b}$. Each preimage $A_{j,b}^e$ is encrypted to a different fresh one-use computation role $P_{j,b}^e$.

At the fixed certification cut, every honest role independently computes the same canonical message M_e and the codeword

$$c_e = \text{Code}(h_{\text{sig}}(M_e)).$$

The protected algorithm of role $P_{j,b}^e$ returns its secret key precisely when $c_e[j] = b$. Observers check **KeyMatch**, decrypt the canonical role packet, and treat the recovered $A_{j,b}^e$ as the opening. A certificate contains valid openings in at least τn coordinates matching the claimed codeword. Corrupt matching roles may withhold, which is why verification does not require every coordinate.

Set $\rho = p$, the computation-role corruption probability of the model, and choose constant margins $\gamma_{\text{live}}, \gamma_{\text{forge}} > 0$ such that

$$1 - \delta(1 - \rho) + \gamma_{\text{forge}} < \tau < 1 - \rho - \gamma_{\text{live}}.$$

The right inequality gives liveness. For a different codeword, every disagreeing coordinate needs the opposite opening, which is available without inverting f only when that opposite role is corrupt; the left inequality prevents enough such openings. The interval is possible when $\delta > \rho/(1 - \rho)$, and hence permits any $\delta > 1/3$ at $\rho = 1/4$. The canonical digest may depend on corrupt-role information visible before its cut. We therefore require the good-assignment event to hold simultaneously for all possible canonical codewords and all ordered pairs of distinct codewords. A conservative sufficient condition is

$$2^\kappa \exp(-\Omega(\gamma_{\text{live}}^2 n)) + 2^{2\kappa} \exp(-\Omega(\gamma_{\text{forge}}^2 \delta n))$$

to be negligible. The exact constants and concentration proof are part of the certificate lemma; they are not claimed proved here.

Speaking exposes the complete state of $P_{j,c_e[j]}^e$, but its branch secret has just become public. The role containing $A_{j,1-c_e[j]}^e$ remains silent forever. This is precisely where the at-most-once rule creates persistent authenticity without a persistent signing key.

The distance-amplified Lamport idea is not new. Katz and Vaikuntanathan use a high-distance encoding in a leakage-resilient one-time signature [12]. We reuse the distance intuition, while the distributed role assignment, partial liveness threshold, post-speech full exposure, and common-input property require a separate argument. Kelsey, Lang, and Lucks study distributed stateful hash-based signing and the disjoint-honest-trustees one-time-key-reuse problem in the absence of broadcast [13].

The certificate is not a signing service for adversarially supplied messages. Honest roles derive M_e from one already closed ledger cut. The common ledger prevents them from being shown different inputs; the certificate makes that public truth usable by an obfuscated program that cannot read the ledger itself.

5.3 Keyless bootstrap and certified commands

The initialization string has the form

$$I = (\text{Init}, \text{uid}, \text{sid}, \text{initDigest}, \text{PKBatch}_0).$$

The program requires the supplied uid to equal the one embedded in U_{ogcs} . The canonical initDigest commits to the initial ledger cut, public initial state, role allocation, and all relevant parameters. The hidden program absorbs the complete encoding of I before deriving its root and emitting Prep_0 , which includes VK_0 and the active labels for the fixed s_0 . Thus initialization is not literally outputless: it emits branch-local encrypted preparation material and public labels for a public state, but no garbled table or application-derived output.

Anyone may run the copyable program on $I' \neq I$. Such a call may even use only adversarial public keys and hence create an adversarially controlled certificate chain. This is harmless only because the different initialization is absorbed before any branch material is derived: every token, label, ciphertext, and later certificate key on that fork lies under a pseudorandomly independent logical root. Later exposure of a canonical role key may decrypt a packet addressed to the same public key on such a fork, but the plaintext contains only fork material. The common ledger selects the live root; no certifying-ledger secret key is needed.

For a public partition of microstep indices into finite intervals J_e , let

$$W_i = H(\text{ogcs}, \text{uid}, \text{sid}, i), \quad i \in J_e,$$

write $\mathbf{W}_e = (W_i)_{i \in J_e}$, and set $\omega_e = \text{Hash}(J_e, \mathbf{W}_e)$. The obfuscated program has no random-oracle access, so these values are computed outside it. Honest certificate roles compute the same vector themselves. The vector supplied with the OGCS call is nonauthoritative: the program checks its digest against the certified ω_e .

Define the full pre-oracle descriptor and its digest by

$$Q_e = (\text{uid}, \text{sid}, e, \text{Hash}(D_{e-1}), \text{Hash}(\text{Prep}_e), \text{inputCheckpoint}_e, \text{PKBatch}_{e+1}), \quad \chi_e = \text{Hash}(Q_e),$$

and let the canonical certificate message be

$$M_e = (Q_e, J_e, \omega_e).$$

The next-key registration cutoff occurs before this message is certified. Exact closed-ledger positions, role allocation, and all domain separators are part of Q_e 's canonical encoding. Thus the program receives, checks, and can use the actual input checkpoint and recipient-key batch rather than only their hash image. It checks the predecessor, epoch, syntax, oracle-vector digest, and certificate under the verification material in its current logical frontier. Any failure enters the absorbing reject state.

The next verification key is an output inside Prep_{e+1} and hence is authenticated by the current accepted extension. It is not included as an independent input to the same command, avoiding a circularity. Different valid subsets or orderings of openings for the same M_e induce the same extension: all derivations use M_e , never the certificate bytes.

5.4 The hidden logical stream

Let InMat_e contain the external-input label pairs, activation tokens, semantic permutations, shares, and role-packet plaintexts underlying Prep_e , and let CertMat_e contain its certificate preimages and packet state. At the beginning of stretch e , the logical frontier is

$$\Phi_e = (X_e, \text{StatePairs}_e, \text{InMat}_e, \text{CertMat}_e, \text{VK}_e),$$

After the public cuts are fixed, let $\overline{\Phi}_e$ be the frontier Φ_e with its seed coordinate replaced by

$$\overline{X}_e = \text{KDF}(X_e, (\text{publicCut}, \chi_e)).$$

Write $\Phi_{e,i}$ for the corresponding intermediate frontier after microstep i , with $\Phi_{e,\ell_e-1} = \overline{\Phi}_e$ and $\Phi_{e,r_e} = \Phi_{e+1}$ on a normally generated stretch. Here $\text{StatePairs}_e = \{(K_{e,w,0}^{\text{st}}, K_{e,w,1}^{\text{st}})\}_w$. The active member of every state pair is public from evaluation of the preceding stretch. For $e = 0$, Prep_0 publishes the member encoding the fixed public initial state; the sibling remains hidden.

Enumerate the finite objects generated for the next suffix by the consecutive indices $J_e = [\ell_e, r_e]$. Set

$$X_{e,\ell_e-1} = \overline{X}_e, \quad X_{e,i} = \text{KDF}(X_{e,i-1}, W_i) \quad (i \in J_e), \quad X_{e+1} = X_{e,r_e}.$$

These equations specify the normal branch. At a programmed microstep, a valid capsule replaces both the emitted fragment and the complete successor microfrontier $\Phi_{e,i}$; subsequent generation continues from that replacement frontier. Domain-separated puncturable-PRF evaluations from these states derive the output-state label pairs and garbling coins for GC_e , followed by InMat_{e+1} , CertMat_{e+1} , verification material, verifiable shares, and role-state encryption coins for Prep_{e+1} . The current InMat_e is not regenerated: it

is the material already committed before the input window. The complete pre-oracle cut is therefore absorbed before the first microstep.

Readout for the newly prepared input roles is bound to the exact ordered recipient-key vector. For example, its coins have the form

$$r_{e+1,w,s,j}^{\text{read}} = \text{PPRF}_{K_{\text{read}}}(X_{e,i}, \chi_e, e+1, w, s, j, pk_{e+1,w,s,j}).$$

Consequently, a canonical packet is tied to its intended recipient key. Packets produced by reset or fake-initialization calls under the same public key are not claimed never to decrypt; rather, prefix isolation must ensure that their plaintexts contain only independently rooted fork material.

The garbling GC_e uses StatePairs_e and the external-input pairs in InMat_e . Its output-state labels are literally the pairs in StatePairs_{e+1} stored in Φ_{e+1} . Evaluation makes only the active member of each pair public. No machine links an old label to a new one, and no committee is needed for carried state.

Changing I , an accepted certified checkpoint, or any earlier W_i changes the logical derivation for the suffix after that point. On the live protocol path, the certificate prevents such a change after a shared state label exists. Calls on another initialization root remain independently evaluable but receive no honest input material belonging to the live root. In a programmed proof hybrid, a sequence of transition capsules may prescribe each affected public fragment together with its matching successor microfrontier; no capsule may program only a table while leaving an inconsistent hidden successor.

5.5 Candidate realization by delayed programming

For a declared input-length bound L , a finite version of the public object may be based on succinct iO for bounded-input Turing machines. Cryptographic iterators and succinct TM-iO provide relevant techniques for compactly representing long bounded computations [14,1]. They do not by themselves instantiate the horizon-free interface above, whose replay input has unbounded length; that requires an additional primitive or a reformulation of the interface. We leave this as a separate realization gap rather than hide L in the environment's running-time polynomial.

For a bounded prefix, we propose to adapt the delayed-backdoor programming of Hofheinz et al.'s universal sampler [11]. Its hidden-trigger paradigm, and the later selectively secure pseudorandom puncturable deterministic encryption (PPDE) of Koppula, Poelstra, and Waters [15], let an apparently random string carry a programmed bounded object. In our proposed adaptation, a random W_i selects the ordinary PRF-generated transition. In a proof, the oracle answer at one punctured point is changed to encode a prescribed transition capsule.

More explicitly, let $(\text{BEnc}, \text{BDec})$ denote the desired puncturable carrier. Its ciphertexts are pseudorandom, while a uniform string decrypts to $\perp$ except with negligible probability. The same hidden microstep program is used in both the real execution and the proof:

$$C_i \leftarrow \text{BDec}_{K_{\text{bd}}}(W_i), \quad \text{Step}_i = \begin{cases} C_i, & \text{if } \text{ValidCap}(C_i, \Phi_{e,i-1}, \chi_e) = 1, \\ \text{NormalGen}(X_{e,i-1}, W_i, \chi_e, i), & \text{otherwise.} \end{cases}$$

On honest random-oracle answers, the normal branch is taken except with negligible probability. In a selective bounded-prefix hybrid in which C_i is fixed before carrier-key setup, the intended proof first replaces the embedded carrier key by a key punctured at C_i and hardcodes the prescribed behavior at the exceptional ciphertext. It separately punctures the normal-generation keys at the same logical point. Koppula–Poelstra–Waters PPDE pseudorandomness can then hide the swap from uniform W_i to $\text{BEnc}_{K_{\text{bd}}}(C_i)$. Choosing C_i online after carrier-key setup requires an adaptive carrier argument and remains open.

The programmed branch bypasses the normal KDF and readout derivations for that microstep and takes the complete successor microfrontier from its capsule. Thus, in the selective experiment, the capsules can be sampled from χ_e , the closed recipient keys, and the prescribed output before W_i or M_e is fixed. This is a specification of the carrier property we need, not a claim that the cited PPDE theorem already supplies the complete stateful interface.

For our use each capsule must contain the *complete* affected logical microtransition: its public garbled fragment and every hidden successor value on which later fragments depend. Programming only one public gate while leaving an inconsistent successor $X_{e,i}$ would not preserve the stream. The intended

hybrid punctures the KDF/PRF path at one canonical microstep, installs this complete capsule, leaves every sibling subtree pseudorandom, and then repeats from left to right.

Two further proof obligations remain visible. First, computational unforgeability of the YAMSO certificate does not make two obfuscated programs functionally identical on every mathematical input. In their binding setup, Fernando et al.’s somewhere statistically unforgeable signatures make alternate valid signatures at a selected round and message statistically nonexistent, except with negligible probability over setup. They use this property to establish functional equivalence of adjacent obfuscated gate programs before applying ordinary iO [6]. This is a candidate technique here, not an instantiation we may assume for free.

Second, the points $H(\text{ogcs}, \text{uid}, \text{sid}, i)$ are predictable after U_{ogcs} is public. A selective-capsule experiment can fix the target sid , finite index set $\bigcup_e J_e$, canonical schedule, and complete capsule sequence before publication, program all corresponding points, and then rely on the fresh uid to rule out an earlier query. This is structural evidence for the bounded carrier hybrid, not a real-ideal simulation: in the protocol the honest inputs and outputs are not known before setup. It also does not give one online simulator for the horizon-free execution.

The intended next pass prepares a fresh unpredictable nonce one stretch ahead. In that proof the simulator first waits for the input and next-key cuts, receives y_e privately, and uses $Z.\text{Complete}$ and $Z.\text{Prepare}$ to form the microstep capsules from χ_e , the recipient keys, and y_e . It then programs the nonce-dependent oracle points, reveals and recomputes $\mathbf{W}_e$, forms M_e , and only then releases the honest openings that take Σ_e across its acceptance threshold. Thus no one can invoke the accepted canonical extension before all of its capsules are installed. We do not repair either issue by giving the environment an explicit time bound.

5.6 What the candidate establishes and what it does not

The construction isolates a plausible path to indefinitely extensible garbling. Adjacent stretches share boundary labels by construction; new public keys enter only through a certified cutoff; external oracle values are bound by a certified digest; and the only online secret holders that speak reveal branch-local state already made public. A fake initialization can generate a whole fake computation, but it forks before any live input or state material.

The missing statements are now explicit: the one-time certificate lemma, a two-stage finite garbling/readout simulation with prescribed boundary labels and full speaker exposure, a horizon-free public-program realization, the stateful delayed-programming lemma, exact iO-compatible authentication, and late online oracle programming. These are organized as whole-process replacement lemmas in section 6. A general universal-streamer abstraction can be revisited after this fixed-function OGCS is understood; the present construction does not claim one.

6 A Modular Proof Outline

The final security statement should be phrased at the application interface. We do not, however, decompose the intermediate argument into a collection of UC ideal functionalities for garbling, OGCS, certificate preparation, and committee readout. A naive decomposition whose interfaces output the secret keys and label pairs needed by the next hybrid would also expose those values to the UC environment. This is the proof-engineering problem encountered when secret key material is passed through intermediate hybrids in the asynchronous YOSO proof [4].

We instead compare two complete interactive processes. They include the environment, adversary, random oracle, ledger, all evaluations of the copyable public OGCS object, every protocol message, and the complete retained state of every role after it speaks. The replacement lemmas below are ordinary indistinguishability statements about these whole executions. A resulting whole-process argument is intended to be used inside a separately constructed top-level UC proof; such a wrapper is not automatic and is not claimed here.

6.1 Scope of the first statement

Before setup, the adversary chooses the corrupt set $\mathcal{C}_{\text{in}}$ of named input providers. This set remains fixed. Each computation registration i is independently marked corrupt by $c_i \leftarrow \text{Bernoulli}(p)$, and an honest

computation role cannot later be corrupted. Nevertheless, immediately after an honest role speaks, its complete role state is given to the adversary. Every computation public key has one canonical role, all outputs are public, and a closed missing or malformed input has the random-default semantics of section 4.

The desired OGCS construction accepts every finite history without a setup-declared horizon. The proof template below therefore assumes a horizon-free online property Y . The bounded-input TM-iO and selective oracle-programming evidence in section 5 establishes neither that property nor a hidden running-time bound; both limitations remain explicit.

6.2 The real and ideal processes

The process $\text{Real}_{\Pi, \mathcal{A}, \mathcal{Z}}(1^\lambda)$ runs the protocol of sections 3 to 5. In addition to the ordinary ledger view, $\mathcal{A}$ and $\mathcal{Z}$ receive

- the complete initial state of every corrupt registered role;
- every public derived garbling, ciphertext, share opening, validity tag, and certificate;
- the result of every adversarial reset or fork evaluation of U_{ogcs} ; and
- the complete secret key, canonical packet plaintext, coins, and retained call record of every honest computation role immediately after its message is irrevocable.

The same disclosed key may retrospectively decrypt packets addressed to it by off-path evaluations of the copyable object; those plaintexts are also part of the real view. Silent honest roles expose nothing and are never reused.

The process $\text{Ideal}_{F, \mathcal{S}, \mathcal{Z}}(1^\lambda)$ evaluates the intended stateful transition directly. Honest named providers give their inputs to the ideal computation; in the first statement every honest provider supplies exactly one bit before its slot is scheduled. For a corrupt provider, $\mathcal{S}$ supplies the semantic bit extracted from its uniquely closed protocol slot, including a bit of the simulator’s choice for a missing or malformed slot. It computes

$$(s_{e+1}, y_e) = F(s_e, a_e)$$

and, once both the input and next-key cuts are fixed, sends $(\text{OutputReady}, e, y_e)$ privately to $\mathcal{S}$. The simulator may use this value internally while it prepares the certified extension, but does not deliver it to $\mathcal{A}$ or $\mathcal{Z}$. The ideal process releases y_e publicly only at the corresponding simulated evaluation event. Thus the environment sees the output at the same point as in the real execution, while $\mathcal{Z}.\text{Complete}$ learns it causally. The ideal process contains no computation PIDs, committees, labels, garbled circuits, random corruption coins, or OGCS state. In the simulation, a default is coupled to $\pi_w^{-1}(0)$; since π_w is uniform, this has exactly the ideal default distribution even though the ideal corrupt-party interface lets the simulator supply the bit.

6.3 Three replacement lemmas

The proof is organized around three properties, denoted X , Y , and Z . They are lemmas about the surrounding execution, not callable ideal functionalities.

X: canonical certification from one-use exposure. Condition on the good certificate-role event. For every epoch, the canonical checkpoint obtains enough valid openings, while no PPT context obtains an accepted certificate for a different checkpoint under the same verification material. This remains true after every role that certified the canonical checkpoint exposes its complete state.

Property X is a non-equivocation-and-liveness statement. Because the canonical digest may depend on corrupt-role information visible before its cut, the good-assignment event is required to hold uniformly for every possible canonical codeword and every ordered pair of distinct codewords. The simulation of encrypted certificate packets, role keys, coins, and exposed call records is assigned to $\mathcal{Z}.\text{Prepare}$ below. The authentication proof separates a collision in h_{sig} , inversion of an unopened one-way image, and an unusually favourable random corrupt-role pattern. With constant margins as in (5.2), a conservative rate-versus-tail condition is

$$2^\kappa \exp(-\Omega(\gamma_{\text{live}}^2 n)) + 2^{2\kappa} \exp(-\Omega(\gamma_{\text{forge}}^2 \delta n)) = \text{negl}(\lambda).$$

Sequential composition then pays a union bound over the polynomial number of certificate epochs appearing in the inspected execution.

Status: this lemma is *conjectured*. Its high-distance Lamport core is source-backed by [12]; the partial opening threshold, common-input restriction, random role assignment, and full post-speech exposure experiment still need a complete proof.

Y: programmable prefix-preserving OGCS. A horizon-free online simulator first produces one public OGCS parameter, before the future initialization, certified descriptors, or outputs are known. When the canonical initialization I arrives, it runs the normal initialization derivation honestly, publishes $D_{-1} = \text{Prep}_0$, and retains the resulting hidden frontier Φ_0 . This step requires no programmed capsule because s_0 is public and there is no application-derived output. Once the pre-oracle descriptor Q_e , the closed PKBatch_{e+1} , and the target microstep capsules for stretch e are available, but before $\mathbf{W}_e$ is exposed, it supports the simulation-side operation

$$\text{Program}(Q_e, \text{Capsules}_e) \longrightarrow \mathbf{W}_e.$$

The canonical message $M_e = (Q_e, J_e, \text{Hash}(J_e, \mathbf{W}_e))$ is formed and certified only afterwards. The resulting object must satisfy

1. every canonical call returns the prescribed output prefix;
2. every accepting history is bit-for-bit prefix consistent, and repeated calls to one history agree exactly;
3. each prescribed capsule contains the complete hidden successor microfrontier as well as its public output fragment;
4. after the first non-equivalent divergence in I , Q_e , J_e , or $\mathbf{W}_e$, arbitrary fork and reset calls reject or evolve under pseudorandomly independent roots; valid certificate-witness variants for the same command are normalized away; and
5. later disclosure of a canonical secret key may decrypt off-path packets addressed to that key, but all such plaintexts are simulatable independent-fork material containing no canonical inactive label or master seed.

The surrounding context may interleave public OGCS evaluations with all ledger activity and speaker exposures.

The candidate bounded-prefix reduction punctures the KDF/PRF path at the next canonical microstep, uses delayed backdoor programming to install the capsule, and replaces sibling subtrees by independent seeds. A separate authentication argument must establish exact or statistical functional equivalence for each iO change; computational certificate unforgeability alone is insufficient.

Status: this lemma is *open*. The cited general-purpose ordinary-iO-based succinct TM-iO gives only a declared bounded-input object. With predictable oracle points, the PPDE evidence applies only to a selective-capsule bounded-prefix experiment in which the capsules are fixed before carrier-key setup, not to the real-ideal simulation above. The hidden-nonce lift at the end of section 5 is the candidate online repair for the latter gaps.

Z: two-stage finite preparation, garbling, and exposure. Property Z has an explicit causal interface.

Z.Prepare. Before the inputs of stretch e are known, fix structural external-input label pairs, token pairs, semantic permutations, public token-validity tags, corrupt shares, honest encrypted role packets, and the complete certificate preparation for that stretch. For $e = 0$, it consumes the honestly derived material returned by Y 's initialization step rather than resampling an incompatible frontier; that material includes full initial-state pairs and the published active member representing s_0 .

Z.Complete(y_e). After the input slots close, take the already fixed active external-input labels, the active carried-state labels, and the public ideal output y_e . Produce garbled tables and decoding information consistent with those labels, together with fresh hidden next-state pairs and their active members. The procedure need not learn the semantic input or carried state. Its fresh preparation call for stretch $e + 1$ uses the already closed PKBatch_{e+1} and pre-oracle descriptor Q_e . This is a simulator-side target sampled without $\mathbf{W}_e$ or M_e ; property Y must subsequently choose or program $\mathbf{W}_e$ so that the public object realizes that target.

Condition on every ordinary readout committee having at most t corrupt members. Initially corrupt members' ciphertexts and shares must be fixed honestly from preparation. When a selected honest role

later speaks, its key, packet, token, coins, shares, and retained call record must be exposed consistently with the reconstructed active label. Encryption hybrids may be used only under honest keys that remain silent. One-role-per-key guarantees one *canonical* packet, not one ciphertext globally: packets created on adversarial fork histories are handled by the isolation clause of Y , while Z simulates their later openings as fork material. Threshold privacy hides the canonical inactive label, and the hidden semantic permutation hides the meaning of the observed structural branch.

The garbling sublemma must support prescribed active external and carried labels, a prescribed public output, and a pair of hidden next-state labels whose active member can be carried into the next simulated stretch. This is stronger syntax than a bare statement of ordinary one-shot garbling privacy. Publishing the tables only after input closure is what makes $Z.\text{Complete}(y_e)$ causal.

Status: this lemma is *conjectured*. Static input-provider corruption is used critically: an honest provider's unopened token map is never exposed after the simulator has selected a dummy structural branch. Token-tag collisions or inversions, sharing failures, and malformed-opening acceptance are recorded separately as auxiliary bad events.

6.4 The straight-line simulator

Assuming X , Y , and Z , the global simulator proceeds as follows.

1. It samples registration and every immutable computation-corruption coin exactly as in the real process and starts the online simulator of Y . When canonical I is fixed, it invokes $Y.\text{Initialize}(I)$ to derive Prep_0 and Φ_0 honestly, then passes the corresponding input and certificate material to $Z.\text{Prepare}$ for stretch 0. No initialization capsule is programmed.
2. For a statically corrupt provider it forms the provider packet and token map honestly. It extracts the effective input from the closed slot. On a malformed or missing slot it supplies $\pi_w^{-1}(0)$ to the ideal process, coupling the uniform protocol and ideal defaults.
3. For an honest provider it publishes one uniformly selected valid structural token. This distribution is independent of the provider's semantic bit, and static corruption means the unused semantic map is never opened. The honest input reaches the ideal process without being shown to the simulator.
4. It fixes corrupt committee shares from preparation. For the selected honest committee it later exposes role states reconstructing the active label; honest sibling roles remain silent.
5. Once the input slots and PKBatch_{e+1} have both closed, it forms the full pre-oracle descriptor Q_e , supplies all extracted corrupt inputs to the ideal process, and privately receives $(\text{OutputReady}, e, y_e)$.
6. It invokes $Z.\text{Complete}(y_e)$ with the active external and carried labels, and invokes $Z.\text{Prepare}$ for stretch $e + 1$ under the closed next-key batch. These outputs determine a finite sequence of complete microstep capsules. Collectively, their public fragments are GC_e , decoding data, and Prep_{e+1} ; their final successor is the entire frontier for stretch $e + 1$. Before an accepted certificate exists, it calls $Y.\text{Program}(Q_e, \text{Capsules}_e)$ to install all of them and obtain $\mathbf{W}_e$.
7. Only after all capsules for the extension are installed does it expose $\mathbf{W}_e$, form M_e , and use X to produce the canonical certificate transcript. In particular, the final honest opening that crosses the acceptance threshold is released after installation. Property X confines the extension to the unique certified checkpoint.
8. Anyone may now evaluate the canonical OGCS extension. Public garbled evaluation exposes the active next-state labels and triggers public delivery of y_e , after which the simulator repeats.

At no point does the simulator receive the ideal hidden state or an honest input bit. It uses only corrupt inputs, prescribed post-speech leakage, and public outputs.

6.5 First bad event rather than circular conditioning

Committee-overflow events are handled first by conditioning on independent GoodCom and GoodCert events and paying their explicit probabilities. Within those conditioned experiments, let BadCrypto be the first occurrence of a token-tag or digest collision, inversion of an unopened one-way value, acceptance of a malformed key or share, a false hidden trigger, or a premature oracle query in the bounded selective programming experiment. Protocol invariants—unique canonical key assignment, absorbing rejection, and exact prefix consistency—are requirements of the construction or property Y , not probabilistic adversarial events.

For each cryptographic bad event, we stop a hybrid immediately before its first occurrence. Adjacent stopped processes are shown indistinguishable; the event is proved negligible in the endpoint where this is easiest; and negligibility is transported back through the hybrids. This is the “Everywhere from Anywhere” pattern of [4, Lemma 8]. The transport loss is retained in the bookkeeping rather than silently discarded.

6.6 Conditional bookkeeping theorem

Let Q_{com} be the number of ordinary size- k readout committees in the inspected prefix. For $p < 1/3$ and $t = \lfloor (k-1)/3 \rfloor$,

$$\delta_{\text{com}} \leq Q_{\text{com}} \exp(-2k(1/3 - p)^2),$$

which is $Q_{\text{com}}e^{-k/72}$ at $p = 1/4$. Let δ_{cert} denote the distinct certificate-role concentration and code-union-bound failure from X . Let $\epsilon_X, \epsilon_Y, \epsilon_Z$ be the three replacement advantages, and let ϵ_{aux} collect only disjoint collision, token-hash, sharing, malformed-opening, and primitive-correctness errors not already included in those terms.

Conditional whole-execution emulation template. If properties X , Y , and Z hold with the horizon-free online quantifiers stated above, then

$$\forall \mathcal{A} \exists \mathcal{S} \forall \mathcal{Z} : \quad \text{Real}_{\Pi, \mathcal{A}, \mathcal{Z}} \approx \text{Ideal}_{F, \mathcal{S}, \mathcal{Z}}.$$

A conservative transported-hybrid accounting has the form

$$\begin{aligned} \text{Adv}_{\Pi, F}^{\text{real-ideal}} &\leq \delta_{\text{com}} + \delta_{\text{cert}} \\ &\quad + 2(\epsilon_X + \epsilon_Y + \epsilon_Z + \epsilon_{\text{aux}}). \end{aligned}$$

The factor two is only a safe placeholder for transporting bad-event bounds; a completed hybrid sequence should replace it by exact accounting.

This is deliberately a theorem *template*, not a claimed security theorem. Property Y is open in both its horizon-free realization and online oracle timing, property Z still requires a concrete two-stage prescribed-label garbling lemma, and property X needs its distributed full-exposure proof. Writing the statement now fixes the interfaces between the arguments without pretending that those arguments have already been completed.

References

1. Ananth, P., Jain, A., Sahai, A.: Indistinguishability obfuscation for turing machines: Constant overhead and amortization. In: Katz, J., Shacham, H. (eds.) *Advances in Cryptology – CRYPTO 2017, Part II*. Lecture Notes in Computer Science, vol. 10402, pp. 252–279. Springer (2017). https://doi.org/10.1007/978-3-319-63715-0_9, <https://eprint.iacr.org/2015/1023>
2. Benhamouda, F., Gentry, C., Gorbunov, S., Halevi, S., Krawczyk, H., Lin, C., Rabin, T., Reyzin, L.: Can a public blockchain keep a secret? In: *Theory of Cryptography*. Lecture Notes in Computer Science, vol. 12550, pp. 260–290. Springer (2020). https://doi.org/10.1007/978-3-030-64375-1_10, <https://eprint.iacr.org/2020/464>
3. Chandran, N., Garay, J., Misra, A.K., Ostrovsky, R., Zikas, V.: Adaptive security, erasures, and network assumptions in communication-local MPC. *Cryptology ePrint Archive*, Paper 2024/1489 (2024), <https://eprint.iacr.org/2024/1489>
4. Damgård, I.B., Kamp, S.H., Loss, J., Nielsen, J.B.: Asynchronous YOSO a la Paillier. *Cryptology ePrint Archive*, Paper 2025/128 (2025), <https://eprint.iacr.org/2025/128>, accepted at TCC; proceedings citation forthcoming
5. Dinsdale-Young, T., Magri, B., Matt, C., Nielsen, J.B., Tschudi, D.: Afgjort: A partially synchronous finality layer for blockchains. In: *Security and Cryptography for Networks*. Lecture Notes in Computer Science, vol. 12238, pp. 24–44. Springer (2020). https://doi.org/10.1007/978-3-030-57990-6_2, <https://eprint.iacr.org/2019/504>
6. Fernando, R., Shi, E., Soni, P., Vanjani, N., Waters, B.: Non-interactive anonymous router with quasi-linear router computation. In: Rothblum, G.N., Wee, H. (eds.) *Theory of Cryptography – TCC 2023, Part III*. Lecture Notes in Computer Science, vol. 14371, pp. 62–92. Springer (2023). https://doi.org/10.1007/978-3-031-48621-0_3, <https://eprint.iacr.org/2022/1395>

7. Gentry, C., Halevi, S., Krawczyk, H., Magri, B., Nielsen, J.B., Rabin, T., Yakoubov, S.: YOSO: You only speak once / secure MPC with stateless ephemeral roles. Cryptology ePrint Archive, Paper 2021/210 (2021), <https://eprint.iacr.org/2021/210>
8. Goldwasser, S., Kalai, Y.T., Popa, R.A., Vaikuntanathan, V., Zeldovich, N.: How to run turing machines on encrypted data. In: Advances in Cryptology – CRYPTO 2013. Lecture Notes in Computer Science, vol. 8043, pp. 536–553. Springer (2013). https://doi.org/10.1007/978-3-642-40084-1_30, <https://eprint.iacr.org/2013/229>
9. Goyal, R., Goyal, V.: Overcoming cryptographic impossibility results using blockchains. Cryptology ePrint Archive, Paper 2017/935 (2017), <https://eprint.iacr.org/2017/935>
10. Guan, J., Korb, A., Sahai, A.: Streaming functional encryption. Cryptology ePrint Archive, Paper 2022/1599 (2022), <https://eprint.iacr.org/2022/1599>
11. Hofheinz, D., Jager, T., Khurana, D., Sahai, A., Waters, B., Zhandry, M.: How to generate and use universal samplers. Cryptology ePrint Archive, Paper 2014/507 (2014), <https://eprint.iacr.org/2014/507>
12. Katz, J., Vaikuntanathan, V.: Signature schemes with bounded leakage resilience. In: Matsui, M. (ed.) Advances in Cryptology – ASIACRYPT 2009. Lecture Notes in Computer Science, vol. 5912, pp. 703–720. Springer (2009). https://doi.org/10.1007/978-3-642-10366-7_41, https://doi.org/10.1007/978-3-642-10366-7_41
13. Kelsey, J., Lang, N., Lucks, S.: Turning hash-based signatures into distributed signatures and threshold signatures: Delegate your signing capability, and distribute it among trustees. IACR Communications in Cryptology **2**(2) (2025). <https://doi.org/10.62056/a6ksudy6b>, <https://doi.org/10.62056/a6ksudy6b>
14. Koppula, V., Lewko, A.B., Waters, B.: Indistinguishability obfuscation for turing machines with unbounded memory. Cryptology ePrint Archive, Paper 2014/925 (2014), <https://eprint.iacr.org/2014/925>
15. Koppula, V., Poelstra, A., Waters, B.: Universal samplers with fast verification. In: Fehr, S. (ed.) Public-Key Cryptography – PKC 2017, Part II. Lecture Notes in Computer Science, vol. 10175, pp. 525–554. Springer (2017). https://doi.org/10.1007/978-3-662-54388-7_18, <https://eprint.iacr.org/2017/012>
16. Lin, H., Pass, R.: Succinct garbling schemes and applications. Cryptology ePrint Archive, Paper 2014/766 (2014), <https://eprint.iacr.org/2014/766>